



ELABORACION DE PAGINAS WEB
INGENIERÍA EN SISTEMAS
IVÁN ANTONIO DE LEÓN FUENTES
ideleonf@miumg.edu.gt

django

django



Modelos



1. Los modelos en Django representan la estructura de datos de la aplicación:
2. Cada modelo corresponde a una tabla en la base de datos.
3. Cada atributo de un modelo representa un campo en la tabla.
4. Se definen en el archivo `models.py` de cada aplicación.

Ejemplo de un Modelo



```
class Proveedor(models.Model):
    nombre = models.CharField(max_length=255, null=True, blank=True)
    contacto = models.CharField(max_length=100, null=True, blank=True)
    direccion = models.CharField(max_length=255, null=True, blank=True)
    nit = models.CharField(max_length=15, null=True, blank=True)
    observaciones = models.TextField(null=True, blank=True)
    habilitado = models.BooleanField(null=True)
    es_producto = models.BooleanField(null=True)
    saldo = models.FloatField(null=True, blank=True)
```

Métodos en los modelos



```
from django.db import models

class Producto(models.Model):
    nombre = models.CharField(max_length=100)
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    stock = models.IntegerField(default=0)

    def aplicar_descuento(self, porcentaje):
        """Aplica un descuento al precio del producto"""
        return self.precio - (self.precio * porcentaje / 100)

    def hay_stock(self):
        """Devuelve True si hay unidades disponibles"""
        return self.stock > 0

    def __str__(self):
        return f"{self.nombre} (${self.precio})"
```

Relaciones entre Modelos



```
class Producto(models.Model): # Renombrado de Medicamento
    codigo = models.CharField(max_length=30, null=True, blank=True)
    nombre = models.CharField(max_length=100, null=True, blank=True)
    nombregenerico = models.CharField(max_length=100, null=True, blank=True)
    descripcion = models.CharField(max_length=100, null=True, blank=True)
    precio = models.FloatField(null=True, blank=True)
    habilitado = models.BooleanField(null=True)
    maximo = models.IntegerField(null=True, blank=True)
    minimo = models.IntegerField(null=True, blank=True)
    composicion = models.CharField(max_length=255, null=True, blank=True)
    presentacion = models.CharField(max_length=255, null=True, blank=True)
    es_producto = models.BooleanField(null=True)
    proveedor = models.ForeignKey(Proveedor, null=True, blank=True, on_delete=models.PROTECT)
    medida = models.ForeignKey(Medida, on_delete=models.PROTECT)
    categoria = models.ForeignKey(Categoria, null=True, blank=True, on_delete=models.PROTECT)
```

Meta Options en los Modelos



1. `db_table`: nombre personalizado de la tabla.
2. `ordering`: orden por defecto en consultas.
3. `verbose_name`, `verbose_name_plural`: nombres legibles.
4. `unique_together`, `constraints`: restricciones.

Migraciones Django



1. Las migraciones aplican cambios en los modelos a la base de datos.
2. Se crean con: `python manage.py makemigrations`
3. Se aplican con: `python manage.py migrate`
4. Se pueden visualizar con: `python manage.py showmigrations`

Consultas con el ORM de Django



1. `Producto.objects.all()`
2. `Producto.objects.filter(precio__gt=100)`
3. `Producto.objects.get(id=1)`
4. `Producto.objects.create(nombre='Nuevo',
precio=50, stock=10)`
5. `Producto.objects.update(precio=80)`
6. `Producto.objects.delete()`